Agentic Systems Engineering

In the previous session, we learned how **Retrieval Augmented Generation (RAG)** adds relevant context to user queries and improves the response quality of LLMs.

RAG made the model ***better informed*** — it could answer questions on topics it didn't learn during training.

The model was still doing exactly one thing: **reading text.**

It could tell you *how* to book a flight.
It could not book the flight.

In this session, we will learn how LLMs can be used to **create plans**, make decisions, and **perform actions** in the real world.

This is the jump from "a model that talks" to "a system that does things"

The agenda

1. What is an AI Agent?
2. The Agentic Loop
3. The ReAct Design Pattern
4. When should you use Agents (and when you shouldn't)

What Is an AI Agent?

An LLM can only output tokens in response to a user query.

A user wants to book a flight:

Q: "Book me a flight from Mumbai to New York."
A: "What dates are you travelling on? Any budget for flight tickets, or other preferences?"

Q: "Tomorrow. $3000–$5000. No hops."
A: "Great. Go to www.jetairways.com to book your flight."

Q: "Book it."
A: "I cannot perform actions on your behalf. However, I can find the best flights."

Q: "..."

The Model gathered requirements, it reasoned about preferences, it even found a website — but the instant the user said "book it," the model hit a wall.

It can describe the action. It cannot take the action.

Tool-use

An LLM can be trained (through fine-tuning) to invoke functions.

Q: "What is the time?"

A: "I am sorry, but as a language model AI, I do not have the capability to access current time or date information. Please check your device for reliable..."

Now here's the same model ***after tool-use training***.

Q: "What is the time?"

"The time is [Calendar("now") = 9:35 AM] 9:35 AM.
You can confirm this at
[searchlink("Get current time") = time.com]
time.com."

Look at what's inside the brackets. The model didn't ***know*** the time — it learned to **ask for it** by producing a call like `Calendar("now")

During tool-use training, the model learns three things:

1. WHEN it should use a tool (a clock question → use a tool, not memory)
2. WHICH tool should be used (Calendar, not a web search)
3. WHAT arguments to pass (Calendar("now"))

That's the entire skill of tool-use: recognizing the boundary of its own knowledge and producing a well-formed request for an external capability.

So is a fine-tuned (tool-using) model an *Agent*?

NO, A tool-use-trained LLM can ***suggest*** making a tool call or executing a web search.

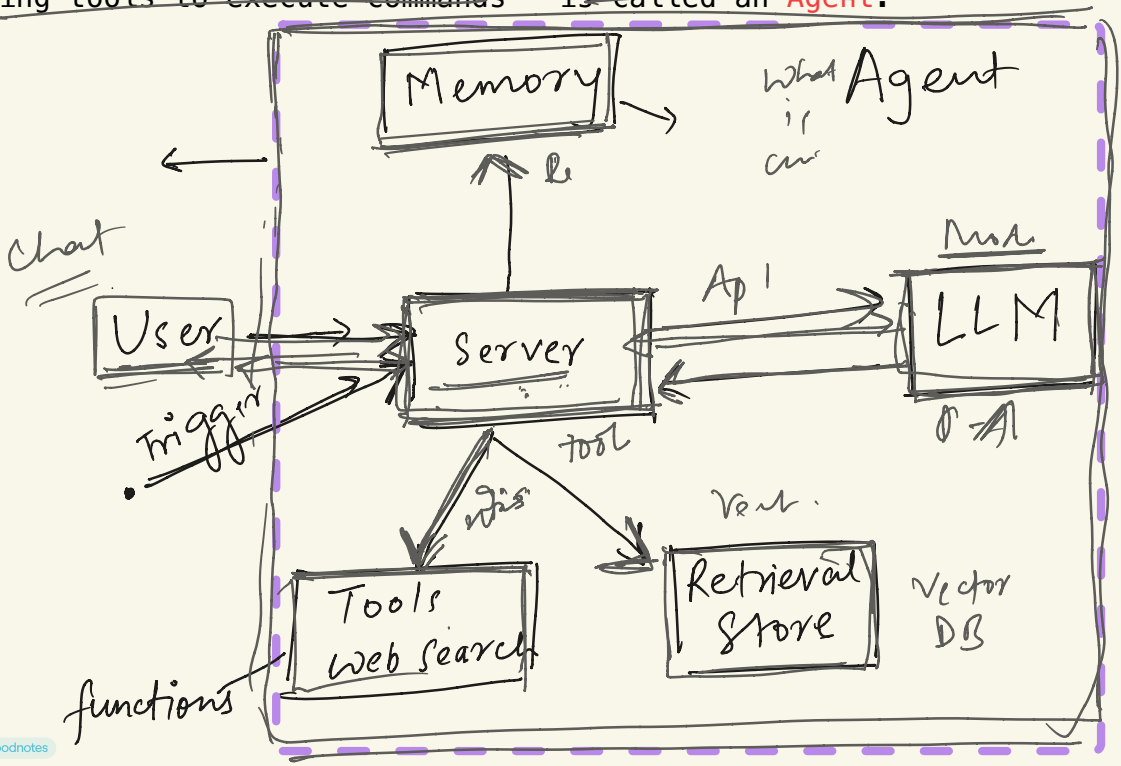
However, it does not call the API or run the function itself.

An LLM needs a system which can execute its instructions and return a response.

Something has to:

1. Read the model's `Calendar("now")` request,
2. *Actually run* the calendar function,
3. Feed the result (`9:35 AM`) back into the model,
4. Let the model continue with that new information.

~~This system – that uses an LLM to plan and make decisions, while using tools to execute commands – is called an Agent~~.



LLM alone = a brain that can only talk

LLM + tools = a brain that knows it should use a tool

Agent = brain + hands + a loop that connects them

An **LLM** is the brain. **Tools** are the hands. An **agent** is the nervous system that lets the brain drive the hands, look at the result, and decide what to do next.

The Agentic Loop

In RAG, the flow was linear and ran exactly once:

query → retrieve context → stuff into prompt → LLM answers → done

An agent is different because it **loops.**

The model takes an action, ***sees the result***, and then decides its next action based on that result.

It keeps going until the task is actually finished — not until it has produced one paragraph of text.

Why does it need to loop?

Because real tasks have steps that depend on each other.

"Book me a flight" requires: search flights → see prices → pick one → check the calendar → reserve a seat → confirm.

GOAL: "Book me a flight to New Delhi tomorrow."

Pass 1 LLM: search_flights("New Delhi", "tomorrow") ← asks for a tool

run it → [3 flights found, cheapest Rs3,200]

↻ a tool was used, so loop again

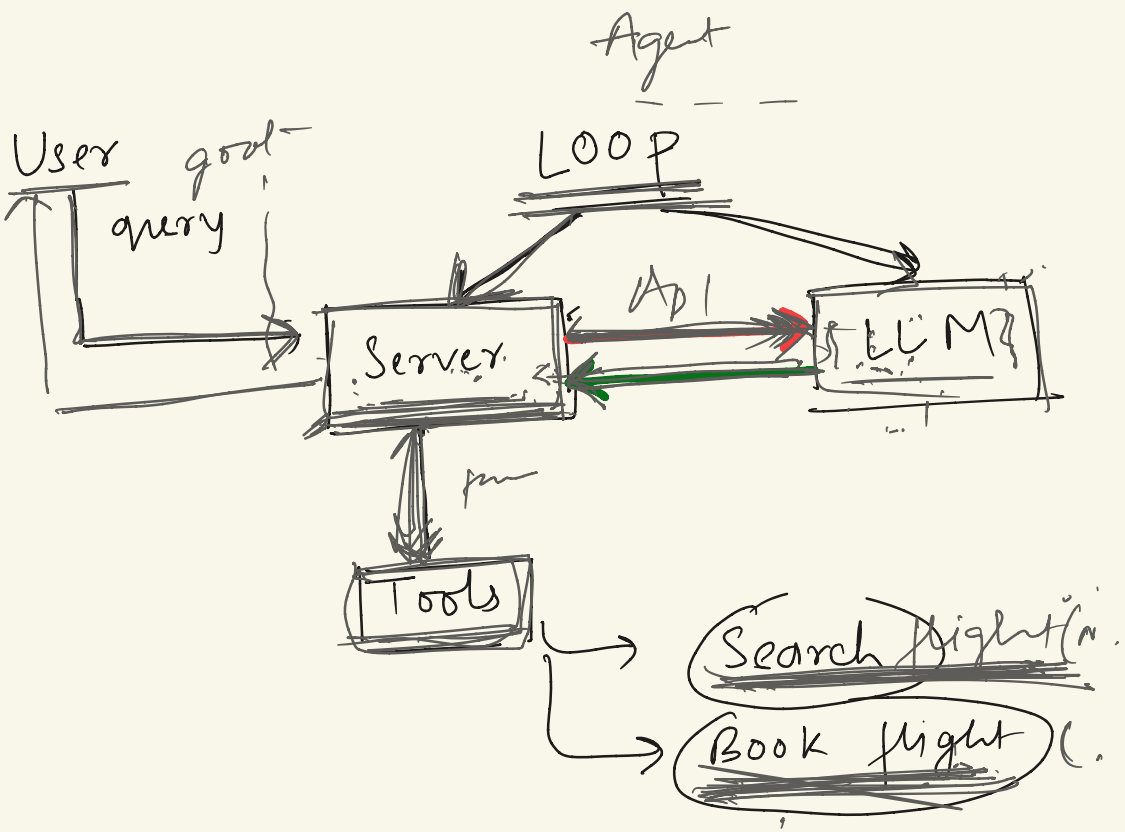
Pass 2 LLM: book_flight(cheapest) ← asks for a tool

run it → {confirmed: "ABC123"}

↻ a tool was used, so loop again

Pass 3 LLM: "Booked! Your confirmation is ABC123." ← no tool

■ no tool requested → STOP. This is the answer.



Whats the exit condition of the loop?

As long as the LLM keeps asking for tools, the loop keeps spinning. The moment it stops asking, you have your answer.

The ReAct Design Pattern

The loop above works, but it has a weakness: the model jumps straight from "goal" to "call this tool" with no visible reasoning.

When it picks the wrong tool, you have no idea ***why***, and neither does the model — it never reasoned about it.

ReAct (short for Reasoning **+ Act**ing**)**

The model produces a **Thought**, then an **Action.**
The system runs the **action** and returns an **Observation.**
Then the model produces its next , and so on.

ReAct in action

Goal: *"*What's the weather in the city I'm flying to tomorrow?*"*

```
Thought:  I don't know which city the user is flying to. I should look
          up their booking first.

Action:   get_booking()

Observation:  { destination: "New Delhi", date: "2026-06-21" }



Thought:  The destination is New Delhi and the date is tomorrow. Now I
          need a weather forecast for New Delhi on that date.

Action:   get_weather(city="New York", date="2026-06-21")

Observation:  { temp_c: 12, condition: "rainy" }



Thought:  I now have everything I need to answer the user. No more
          tools required.

Action:   finish("It'll be about 12°C and rainy in New Delhi
tomorrow.")
```

ReAct vs. the raw loop

It makes the model more reliable.

Defining tools: what the LLM actually receives

We keep saying the LLM "asks for a tool" — but how does it know which tools exist in the first place?

You have to define them.

A tool definition is a JSON schema describing the tool's name, what it does, and the arguments it accepts.

Here's a tool that books a flight:

```json
{
  "name": "book_flight",
  "description": "Call this to execute a booking ONLY after a flight_id and passenger info are confirmed.",
  "parameters": {                                  // <- passed into the tool as arguments
    "type": "object",
    "properties": {
      "flight_id":       { "type": "string", "description": "Unique ID from the search results." },
      "passenger_name":  { "type": "string" },
      "passport_number": { "type": "string" }
    },
    "required": ["flight_id", "passenger_name", "passport_number"]
  }
}
```

The name, description, and input parameters** of every tool must be defined in this schema.

The model reads *only this* — it never sees your actual Python function. The schema is the entire contract.

The description field is where agents are won or lost

The model decides *when* and *how* to call a tool almost entirely from the `description`.

A vague description ("books a flight") produces an agent that calls the tool at the wrong time with the wrong arguments.

Treat the description as a mini spec.

TIP — put all five of these in the description field:

> 1. **What** does this tool do?
> 2. **When** is it used? (triggers, conditions — note our example says "ONLY after ... confirmed")
> 3. **What each parameter means.**
> 4. **What** the tool **returns.**
> 5. **Examples** of input parameters.

Where the schema actually goes: into the System prompt

Langchain Tutorial -https://www.linkedin.com/pulse/honest-guide-langchain-beginners-vinay-c-iyojc/